

Complexity Analysis

Complexity analysis is the theoretical evaluation of how much resources an algorithm consumes as the input size (n) grows

In it we want to achieve **platform-independence**:

Use of an abstract machine that uses steps of time & units of memory instead of seconds or bytes.

- each **elementary operation** takes 1 step
- each **elementary instance** ^{copy} takes 1 unit of memory

TWO dimensions of complexity:

- 1) Time complexity
- 2) Space complexity

Time complexity:

Standard Analysis techniques:

- 1) Constant time statements
- 2) Analyzing loops
- 3) Analyzing nested loops
- 4) Analyzing sequence of statements
- 5) Analyzing Conditional Statement.

Now we will start ~~solving~~ finding complexities of algorithms ~~but~~ & in them we will find a cert function & ~~might~~ Big-Oh (worst case).

1) Analyzing an Algorithm:

consider question:

```
int sum ( int A[], int N) {
```

```
    int s = 0;
```

```
    for (int i = 0; i < N; i++)
```

```
        s = s + A[i];
```

```
    return s;
```

```
}
```

Now to find its cost function $f(n)$ we need to look at all of its operations.

```
int sum ( int A[], int N) {
```

```
    int s = 0; — once
```

```
    for (int i = 0; i < N; i++)
```

```
        s = s + A[i]
```

N times N times N times (reference).

```
    return s; — once
```

```
}
```

so label:

$$T(n) = 5N + 4.$$

why did condition $i < N$ run $N+1$ times?

when loop runs, it runs & condition fulfills N times but fails 1 time but in all this condition is checked $\frac{N}{1} + \frac{1}{1}$ times.
Success attempts Fail attempts.

all other things inside loop & it will run N time because they only run when condition is true.

for big-O : ignore constants & so

$O(N)$

so for big-O notation we can even do it quickly without finding cost function because it ignore most details.

Imp points:

- Declaration alone (int j) is considered of O cost. Assignment operation has 1 cost.

2) Analyzing Simple loops:

Every loop has 2 parts:

- 1) How many iterations are performed?
- 2) How many steps per iteration?

e.g.

```
int sum = 0, j;
```

```
for (j=0; j<N; j++)
```

```
    sum = sum + j;
```

2 steps.

- ~~loop~~ iterations = N .

- steps per iteration = $2 = O(1)$.

in big-O

so $N * O(1) = O(N)$.

Q.

```
int sum = 0;
for (i = 1; i < n; i = i + 2)
    sum = sum + i;
```

2 steps.

iterations = $N/2$
 # of steps per iteration = 2 = $O(1)$

$$\frac{N}{2} * O(1) = O\left(\frac{N}{2}\right) = O(N)$$

constants ignored.

Question:

```
int sum = 0
for (i = 1; i < 100; i = i + 1)
    sum = sum + i;
```

~~100~~ $100 * O(1) = O(100) = O(1)$

3) Analyzing Nested Loops:

For this, you should treat each loop like a single loop & evaluate each level of nesting as needed.

- In this typically ignore, any body of a loop for cost function & we only include:

- Conditional & of loop
- the inner most body steps.

As we go down the loops in nesting we need to multiply its ~~body~~ own ~~com~~ complexity with the amount of time its previous loops are calling it.

see example on next page.

Example 1:

```

int j, k;
for (j=0; j<n; j++)
    for (k=0; k<n; k++)
        sum += k+j;

```

$n+1$ times.
 $n * (n+1)$ times
 own conditional statement complexity
 The outer loop runs its body n times so multiplies it here.
 n
 For loop body runs n times
 from outer loop.

$$f(n) = (n+1) + (n * (n+1)) + (n * n)$$

$$= n+1 + n^2 + n + n^2$$

$$F(n) = 2n^2 + 2n + 1$$

$$F(n) = O(n^2).$$

as the loop nesting increases number of multipliers of a statement will also increase.

Example 2:

```

for (i=0; i < n; i++) {
    for (j=0; j < n; j++) {

```

$c[i, j] = 0$; $\rightarrow$ ignored as only inner most body is considered only.

```

        for (k=0; k < n; k++) {

```

```

             $c[i, j] = c[i, j] + A[i, k] + B[j, k];$ 
        }
    }
}

```

$n * n * n$

$$F(n) = (n+1) + (n * (n+1)) + (n * n * (n+1)) + n^3$$

$$= n+1 + n^2 + n + n^3 + n^2 + n^3$$

$$F(n) = 2n^3 + 2n^2 + 2n + 1$$

$$F(n) = O(n^3)$$

Example 3:

```
for (j=0; j<N; j++) {  
    for (k=N; k>0; k--) {  
          
    }  
}
```

doesn't matter step up or down
remains same so
 $O(N^2)$.

Example 4:

```

for (i = 1; i <= n; i++) {
    for (j = 1; j <= i; j++) {
        statement;
    }
}

```

For question like these try to do some iterations:

i	j	Number of iterations of j
i = 1	j = 1	j <= i ✓ yes j++
	j = 2	j <= i ✗ No
i = 2	j = 1	j <= i ✓ yes j++
	j = 2	j <= i ✓ yes j++
	j = 3	j <= i ✗ No
i = 3	j = 1	j <= i ✓ yes j++
⋮	⋮	⋮

so by this we understand total number of iterations/steps for the inner loop:

number of runs: 1, 2, 3, 4, 5 -----
 i=1 i=2 i=3 i=4 i=5

Now if we see this with basis of n , it becomes a sum:

$$1 + 2 + 3 + 4 + \dots + N$$

sum of arithmetic series:

$$\text{so } O\left(\frac{N * (N+1)}{2}\right) = O(n^2)$$

so for these Questions if we can find a series then that sum of series allows us to find the complexity.

4) Analyze when Index is not linear:

Example 1:

```
i = 1  
while (i < n) {  
    i = 2 * i;  
}
```

we need to see ~~to~~ & find a pattern/sense to understand these problems.

first make a table of iterations:

~~i = 1~~
~~i = 1~~ i = 1 ; i < n ;
i = 2 ; i < n ;
i = 4 ; i < n ;
i = 8 ; i < n ;
i = 16 ; i < n ;
i = 32

so pattern followed is:

$$2^0, 2^1, 2^2, 2^3, 2^4, \dots$$
$$i = 2^k$$

2 steps)

- 1) Find how is i changing (already done)
- 2) See the stopping condition.

in code it stops when
 $i \geq n$

so

~~2~~

$$2^k = n \quad \# \text{ as } i \geq n \text{ so } i = 2^k$$

$$k = \log_2(n).$$

$$F(n) = O(\log(n)).$$

Example 2:

$$p = 0;$$

for ($i = 0$; $p < n$; $i++$) {

$$p = p + i;$$

}

for such questions :

1) Find i & p relationship at the end with n as the number of iterations.

2) Then seeing that we ~~know~~ know $p < n$, so by equation p in terms of i we get i in relation with n & we can calculate complexity.

$i = 1$	$p = 1$	$(0 + 1)$
$i = 2$	$p = 3$	$(0 + 1 + 2)$
$i = 3$	$p = 6$	$(0 + 1 + 2 + 3)$
$i = 4$	$p = 10$	$(0 + 1 + 2 + 3 + 4)$

day/date

so

$$p = \frac{i(i+1)}{2}$$

now

~~$p = \frac{i(i+1)}{2}$~~

in Big-O we ignore constants

so

$$p = i^2$$

This is to make it easy.

now substitute i into condition

$$p < n$$

$$i^2 < n$$

$$i < \sqrt{n}$$

so

$$F(n) = O(\sqrt{n})$$